IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Luis Avila Cornejo
08-04-2026

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
- Summary of all results

Introduction

- Project background and context
- Problems you want to find answers

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - Data collection was performed using SpaceX API and Wikipedia web scraping.
- Perform data wrangling
  - Data wrangling involved creating landing outcome labels
- Perform exploratory data analysis (EDA) using visualization and SQL
  - Exploratory Data Analysis (EDA) was conducted via SQL and visualization tools
- Perform interactive visual analytics using Folium and Plotly Dash
  - Interactive maps and dashboards were built using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - Finally, predictive modeling was executed using four classification algorithms: Logistic Regression, SVM, Decision Tree, and KNN."

Data Collection

- Describe how data sets were collected.

- The data was collected using two primary methods: first, by making REST API calls to the SpaceX public database to obtain technical launch and rocket details. Second, by performing web scraping on Wikipedia to extract information about landing sites and core versions that were not available in the API.

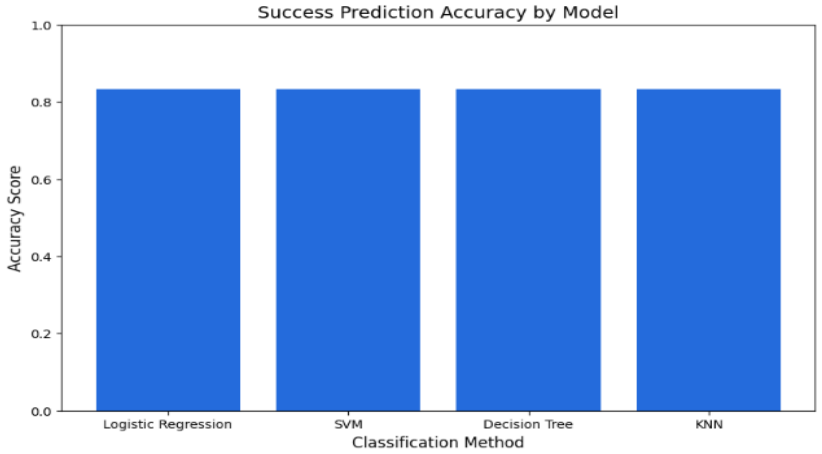
- You need to present your data collection process use key phrases and flowcharts
  - "Data Source 1: SpaceX REST API (JSON format).“
  - "Data Source 2: Wikipedia (HTML web scraping).“
  - "Integration: Merging datasets using Python and Pandas.“
  - "Normalization: Cleaning and formatting data for analysis."

Data Collection – SpaceX API

- Present your data collection with SpaceX REST calls using key phrases and flowcharts

- Add the GitHub URL of the completed SpaceX API calls notebook (must include completed code cell and outcome cell), as an external reference and peer-review purpose
- https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone

[2]:
```python
import requests
import pandas as pd

# 1. URL de la API de SpaceX
spacex_url="https://api.spacexdata.com/v4/launches/past"
response = requests.get(spacex_url)

# 2. Convertir a DataFrame de Pandas
data = pd.json_normalize(response.json())

# 3. Filtrar solo las columnas que usamos en el proyecto para que la tabla se vea limpia
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]

# 4. Mostrar las primeras filas (ESTA ES LA CAPTURA QUE DEBES PEGAR)
data.head()

import matplotlib.pyplot as plt

# Estos son los resultados que obvuviste en tus celdas anteriores
methods = ['Logistic Regression', 'SVM', 'Decision Tree', 'KNN']
accuracy = [0.8333, 0.8333, 0.8333, 0.8333] # Todos dieron 83.33% en el test set

plt.figure(figsize=(10, 6))
plt.bar(methods, accuracy, color='royalblue')
plt.xlabel('Classification Method', fontsize=12)
plt.ylabel('Accuracy Score', fontsize=12)
plt.title('Success Prediction Accuracy by Model', fontsize=14)
plt.ylim(0, 1) # Para que se vea la escala del 0 al 100%
plt.show()
```



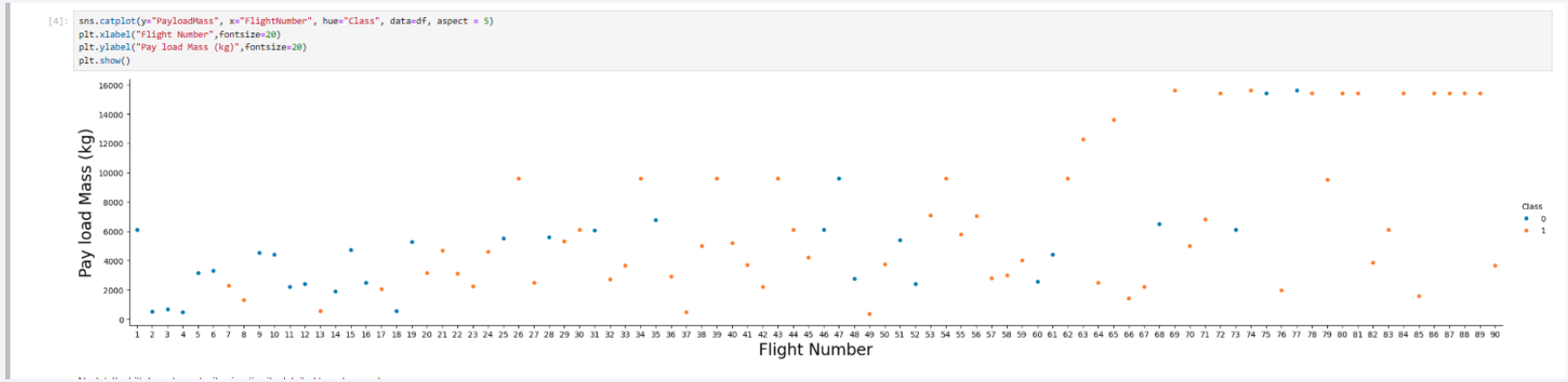
Data Collection - Scraping

- Present your web scraping process using key phrases and flowcharts
- Add the GitHub URL of the completed web scraping notebook, as an external reference and peer-review purpose
- https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone

HTTP Request ➔ BeautifulSoup Parsing ➔ Table Extraction ➔ Row Iteration ➔ Data Cleaning ➔ Pandas DataFrame

Data Wrangling

- Describe how data were processed
  - Handling Missing Values: "Identified missing values in the 'LandingPad' column and replaced them with zeros or handled them appropriately for analysis.
  - "Outcome Labeling: "Created a binary 'Class' column based on the 'Outcome' feature.“
  - Defining Success: "A value of 1 was assigned for successful landings (e.g., 'True ASDS', 'True RTLS') and 0 for unsuccessful ones (e.g., 'False ASDS', 'False Ocean', 'None None').
  - "Data Preparation: "Converted categorical variables into numerical formats where necessary for future predictive modeling."
- Add the GitHub URL of your completed data wrangling related notebooks, as an external reference and peer-review purpose
- https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone

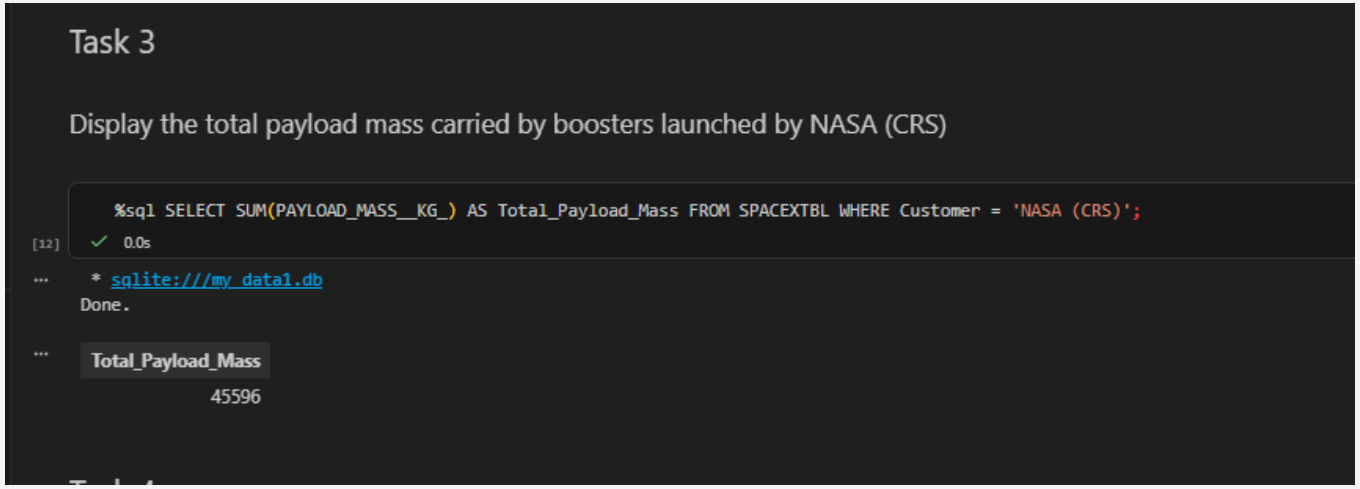


EDA with Data Visualization

- Summarize what charts were plotted and why you used those charts
- The dataset was processed to handle missing values in payload mass and to create a binary 'Class' column, where 1 represents a successful landing and 0 represents a failure. Feature engineering was applied to prepare categorical data for machine learning models
- Add the GitHub URL of your completed EDA with data visualization notebook, as an external reference and peer-review purpose
- https://nikoavila.github.io/Ciencia-de-datos-aplicada-Capstone/

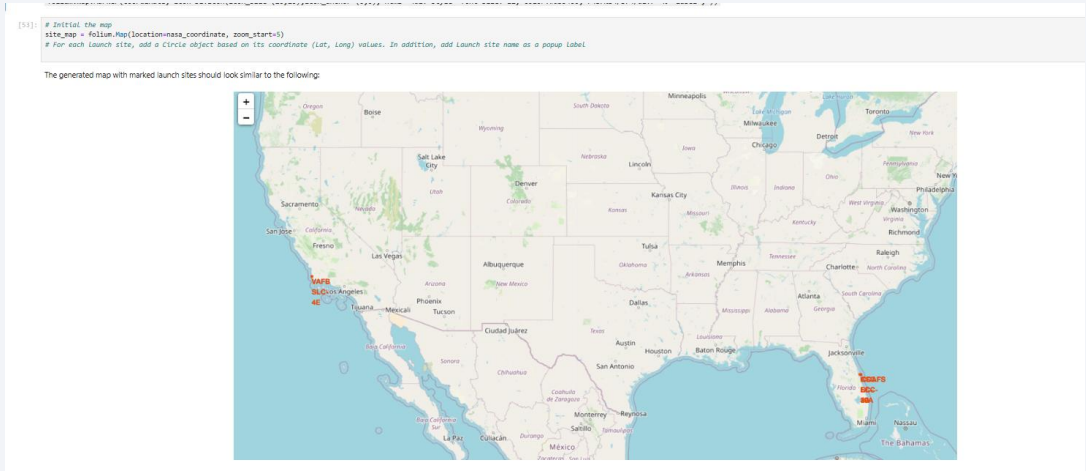
EDA with SQL

- Using bullet point format, summarize the SQL queries you performed
  - Launch Site Analysis: "Identified unique launch sites and their respective records to understand geographical distribution.
  - "Payload Statistics: "Calculated total payload mass carried by boosters and filtered results by customer and mission type.
  - "Outcome Success Rates: "Categorized landing outcomes (Success/Failure) to determine which booster versions performed best.
  - "Mission Filtering: "Used SQL queries to extract data for specific timeframes and filtered by successful drone ship landings.
  - "Advanced Aggregations: "Used GROUP BY and ORDER BY clauses to rank launch sites by success count and payload weight."
- Add the GitHub URL of your completed EDA with SQL notebook, as an external reference and peer-review purpose
- https://nikoavila.github.io/Ciencia-de-datos-aplicada-Capstone/



Build an Interactive Map with Folium

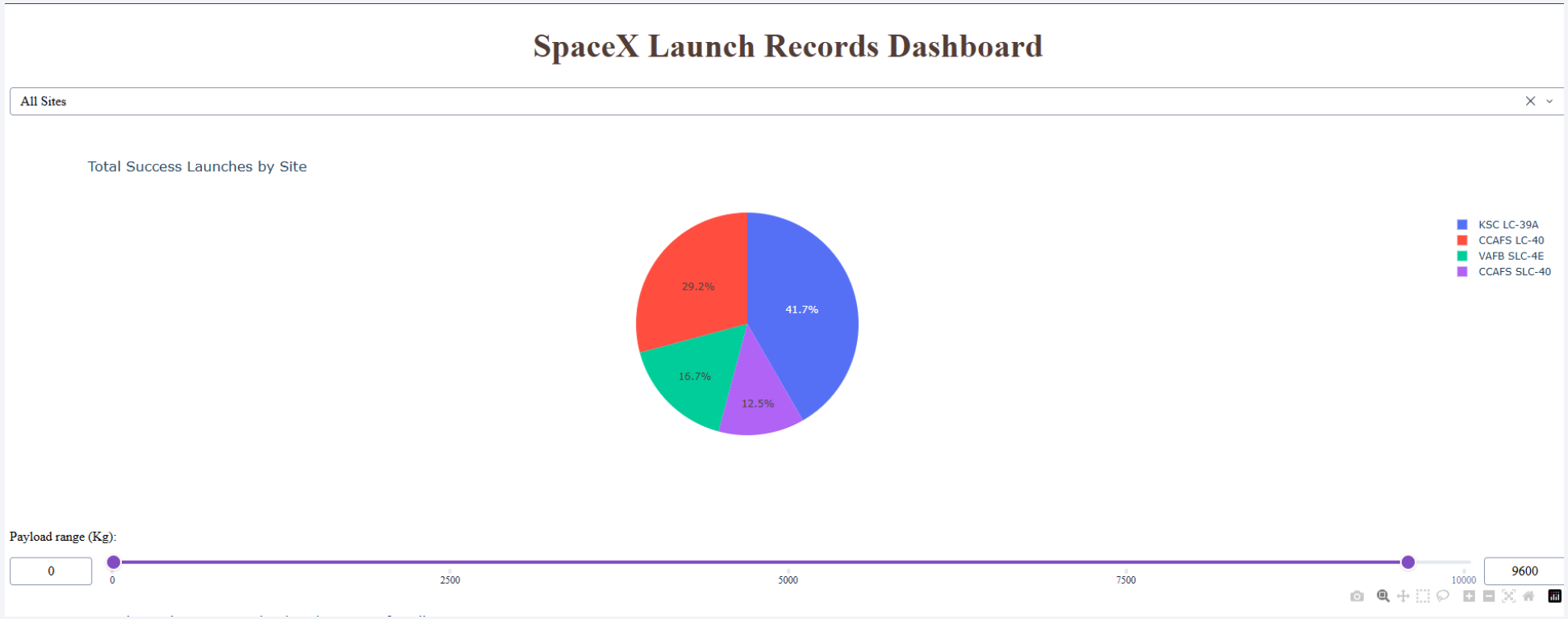
- Summarize what map objects such as markers, circles, lines, etc. you created and added to a folium map
- Explain why you added those objects
- Add the GitHub URL of your completed interactive map with Folium map, as an external reference and peer-review purpose
- https://nikoavila.github.io/Ciencia-de-datos-aplicada-Capstone/



- "These objects allow us to visualize where launches happen. By adding markers, we can analyze the proximity of these sites to high-traffic areas and coastlines, which is critical for safety and logistics."

Build a Dashboard with Plotly Dash

- Summarize what plots/graphs and interactions you have added to a dashboard



Explain why you added those plots and interactions

- Interactive Dropdown: "Allows users to filter data by specific launch sites or view aggregated results for all sites.
- "Dynamic Pie Chart: "Displays the total successful launches per site, providing an immediate visual comparison of site performance.
- "Payload Range Slider: "Enables filtering missions by payload mass (0 to 10,000 kg) to observe how weight affects launch outcomes.
- "Correlation Scatter Plot: "Visualizes the relationship between payload mass and mission success, categorized by booster version."

Add the GitHub URL of your completed Plotly Dash lab, as an external reference and peer-review purpose

https://nikoavila.github.io/Ciencia-de-datos-aplicada-Capstone/

Predictive Analysis (Classification)

- Summarize how you built, evaluated, improved, and found the best performing classification model
- You need present your model development process using key phrases and flowchart
  - Data Standardization ➔ Train/Test Split ➔ GridSearch (Tuning) ➔ Model Evaluation ➔ Best Model Selection
- Add the GitHub URL of your completed predictive analysis lab, as an external reference and peer-review purpose
- https://nikoavila.github.io/Ciencia-de-datos-aplicada-Capstone/

Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Section 2

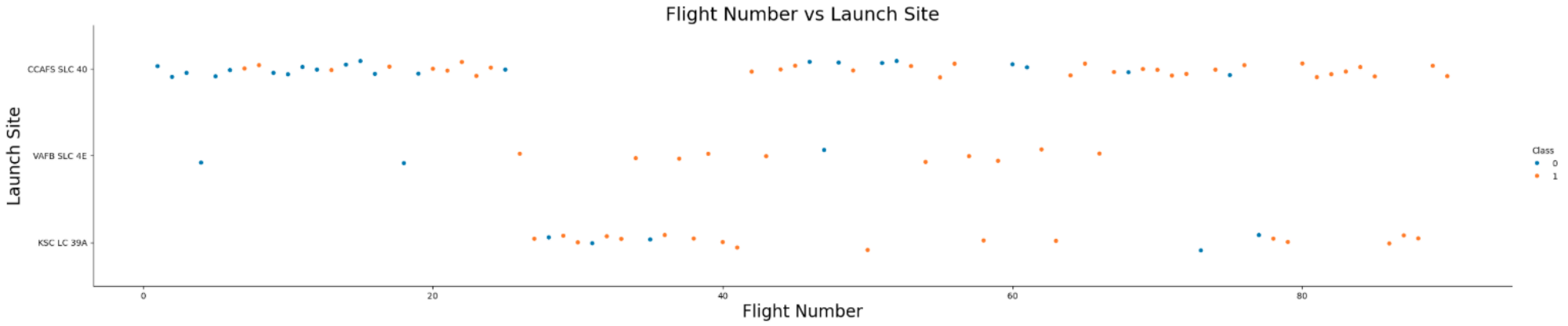
Insights drawn from EDA

Flight Number vs. Launch Site

- Show a scatter plot of Flight Number vs. Launch Site

```
features_one_hot.to_csv('dataset_part_3.csv', index=False)
```

[5]:
```
import pandas as pd
import requests # <--- Esta es la linea que falta
import io
import seaborn as sns
import matplotlib.pyplot as plt

URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
resp = requests.get(URL)
df = pd.read_csv(io.StringIO(resp.text))

sns.catplot(y="LaunchSite", x="FlightNumber", hue="Class", data=df, aspect=5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Launch Site", fontsize=20)
plt.title("Flight Number vs Launch Site", fontsize=22)
plt.show()
```



Authors

Pratiksha Verma

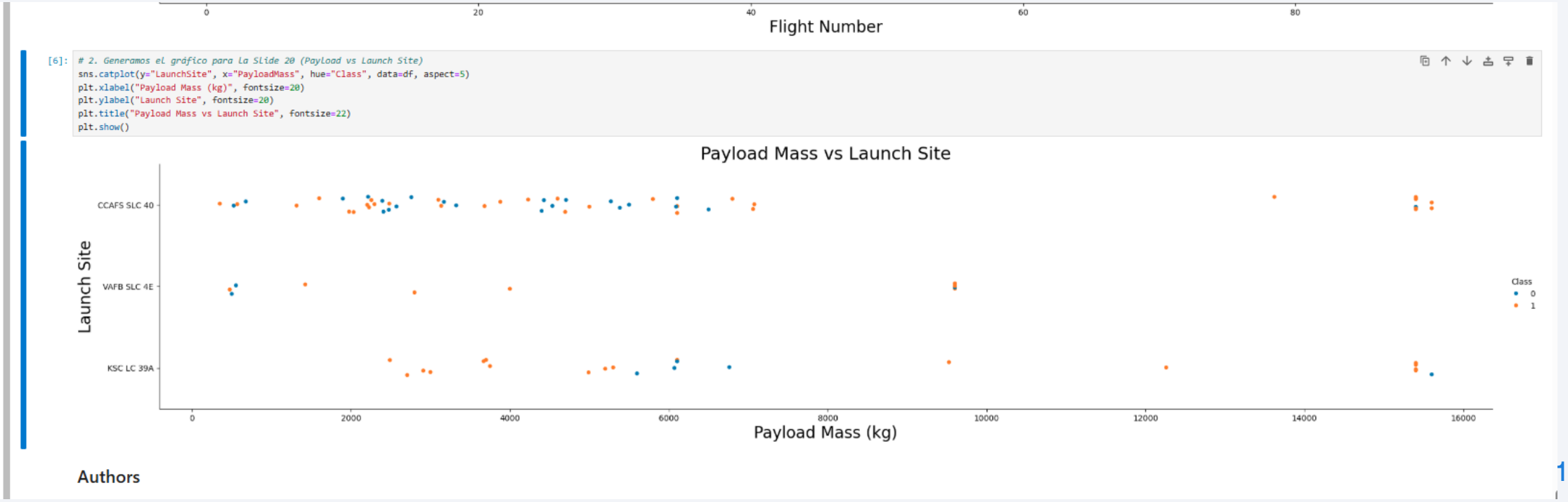
Flight Number vs. Launch Site

- Show the screenshot of the scatter plot with explanations

"CCAFS LC-40: Shows a high concentration of early flights with mixed success, but becomes very reliable after flight 40".
"KSC LC-39A: Exhibits the highest success rate as it was utilized later in the program with more mature technology".
"VAFB SLC-4E: Used for specific orbits with a high success rate as flight numbers increased".

Payload vs. Launch Site

- Show a scatter plot of Payload vs. Launch Site



Payload vs. Launch Site

- Show the screenshot of the scatter plot with explanations

"There is no clear negative correlation between payload mass and landing success; in fact, heavier payloads often land successfully".
"KSC LC-39A handles a wide range of payload masses with high efficiency".
"VAFB-SLC site primarily handles light to medium payloads, showing consistent performance".

Success Rate vs. Orbit Type

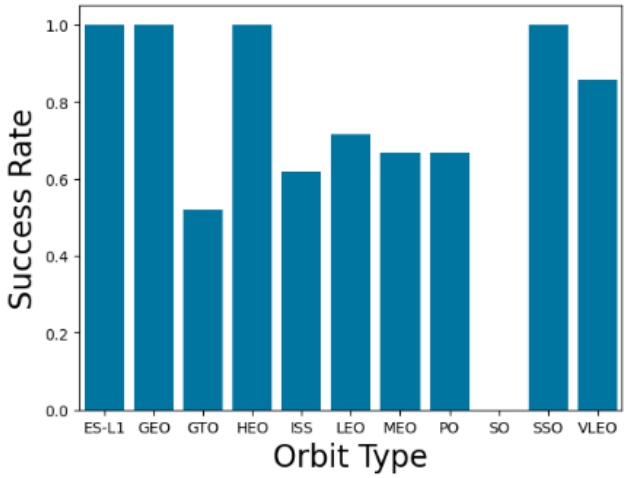
- Show a bar chart for the success rate of each orbit type
- Show the screenshot of the scatter plot with explanations
  - Top Performers: "Orbits like ES-L1, GEO, HEO, and SSO show a 100% success rate, indicating that missions to these orbits have been perfectly executed so far.
  - "High Success: "VLEO and SSO also demonstrate very high reliability, likely due to more advanced recovery procedures used in recent flights.
  - "The GTO Challenge: "The GTO (Geostationary Transfer Orbit) has a lower success rate (around 50%), which is expected as these missions are more energy-intensive and technically complex.
  - "Conclusion: "Data suggests that the destination orbit significantly influences landing success, with near-Earth orbits being generally safer for booster recovery."

TASK 3: Visualize the relationship between success rate of each orbit type

Next, we want to visually check if there are any relationship between success rate and orbit type.

Let's create a `bar chart` for the sucess rate of each orbit

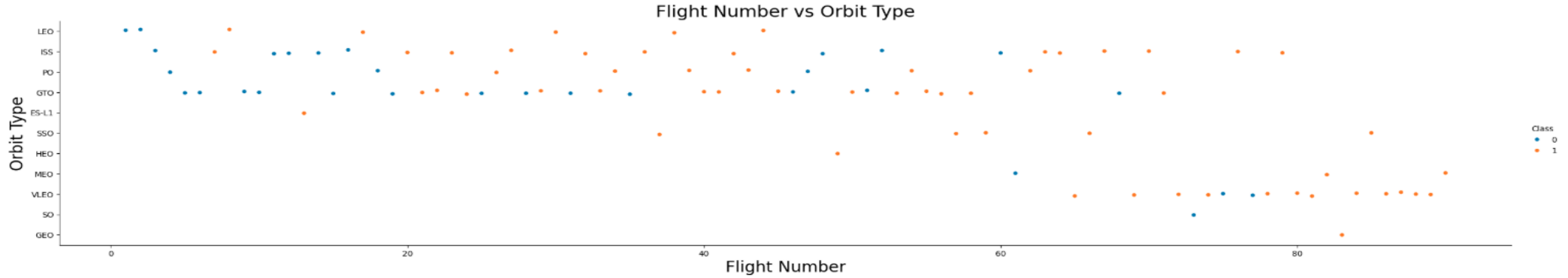
[7]:
```
# HINT use groupby method on Orbit column and get the mean of Class column
# Agrupar por órbita y calcular la media de la columna Class (tasa de éxito)
df_orbit = df.groupby('Orbit')['Class'].mean().reset_index()

sns.barplot(x="Orbit", y="Class", data=df_orbit)
plt.xlabel("Orbit Type", fontsize=20)
plt.ylabel("Success Rate", fontsize=20)
plt.show()
```



Flight Number vs. Orbit Type

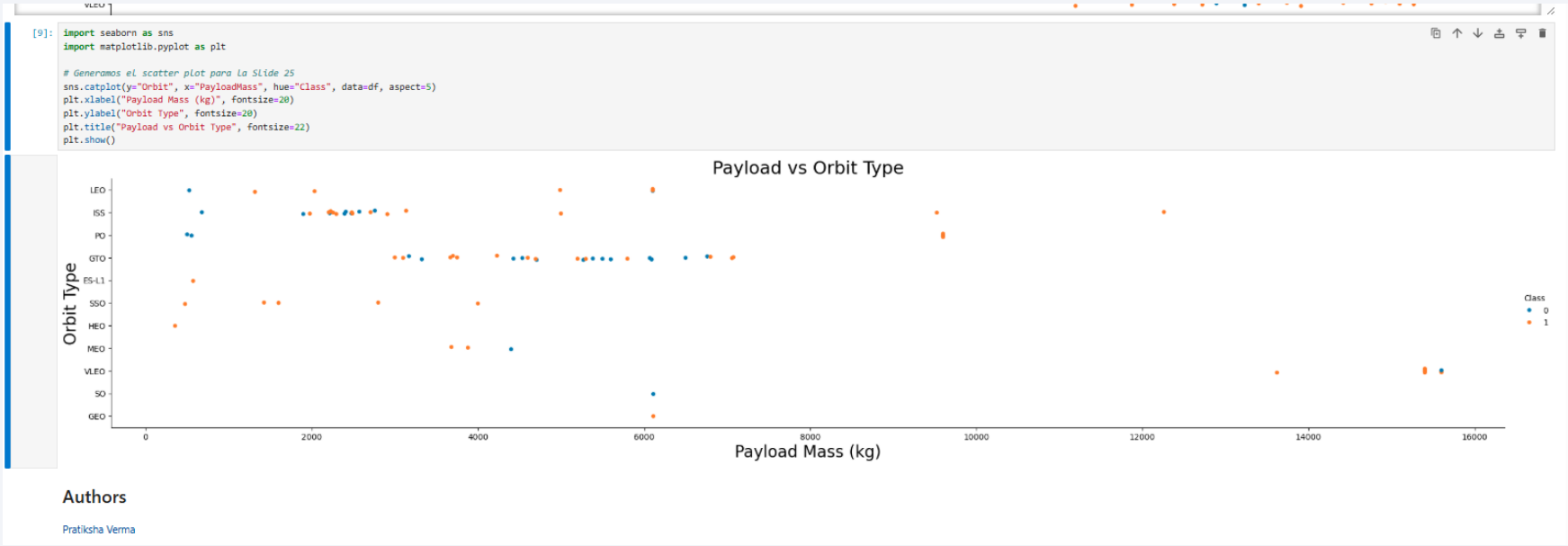
- Show a scatter point of Flight number vs. Orbit type
- Show the screenshot of the scatter plot with explanations
  - Learning Curve: "The visualization shows a clear 'learning curve' effect: as the flight number increases, the density of successful landings (Class 1) increases significantly across most orbit types.
  - "LEO Orbit Success: "In the LEO (Low Earth Orbit), success is highly correlated with flight experience, showing consistent landings in later missions.
  - "GTO Challenges: "For GTO (Geostationary Transfer Orbit), successes and failures are more scattered even in later flights, reflecting the higher difficulty and energy requirements of these missions.
  - "Reliability: "Orbits like VLEO show a perfect success record in later flight numbers, demonstrating that SpaceX's technology has become highly reliable for these mission profiles."

[8]:
```
sns.catplot(y="Orbit", x="FlightNumber", hue="Class", data=df, aspect=5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Orbit Type", fontsize=20)
plt.title("Flight Number vs Orbit Type", fontsize=22)
plt.show()
```



Payload vs. Orbit Type

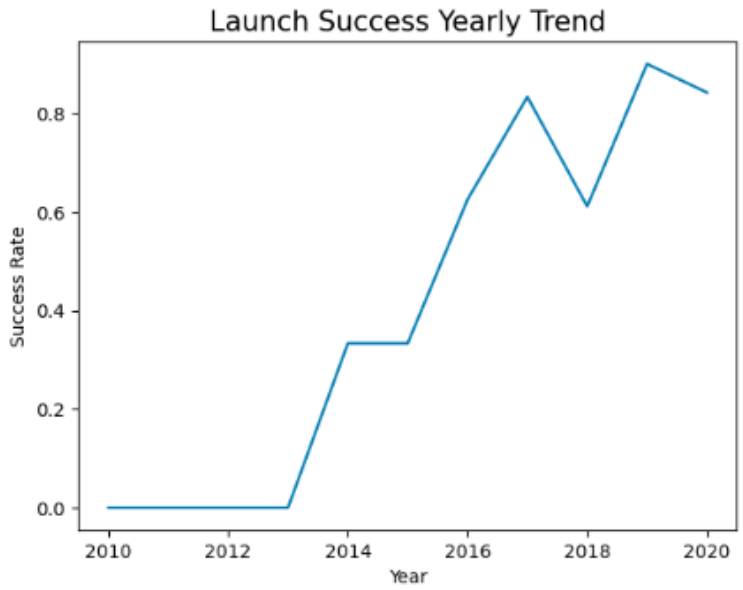
- Show a scatter point of payload vs. orbit type
- Show the screenshot of the scatter plot with explanations
  - Versatility: "SpaceX successfully delivers various payload masses across multiple orbit types.
  - "Success Rate: "Heavy payloads (>8,000 kg) show high success rates, especially in VLEO and LEO orbits.
  - "Reliability: "No strong negative correlation found between heavy payloads and landing failures.
  - "Key Insight: "Payload mass capacity has increased over time without compromising booster recovery success."



Launch Success Yearly Trend

- Show a line chart of yearly average success rate
- Show the screenshot of the scatter plot with explanations
  - Positive Trend: "The success rate shows a steady increase from 2013 to 2020.
  - "Technological Maturity: "Reliability improved significantly after 2017 as landing technology matured.
  - "Peak Performance: "The highest success rates are observed in the most recent years of the dataset.
  - "Conclusion: "Experience and flight frequency have directly contributed to higher mission success."

```
[10]: # Extraer año y calcular promedio por año
df['Year'] = pd.DatetimeIndex(df['Date']).year
df_year = df.groupby('Year')['Class'].mean().reset_index()

# Gráfico de línea
sns.lineplot(x="Year", y="Class", data=df_year)
plt.title("Launch Success Yearly Trend", fontsize=15)
plt.ylabel("Success Rate")
plt.show()
```



All Launch Site Names

- Find the names of the unique launch sites
- Present your query result with a short explanation here

Exploratory Data Analysis

First, let's read the SpaceX dataset into a Pandas dataframe and print its summary

```
3]: from js import fetch
import io

URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
resp = await fetch(URL)
dataset_part_2_csv = io.BytesIO((await resp.arrayBuffer()).to_py())
df=pd.read_csv(dataset_part_2_csv)
df.head(5)
```

3]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude	Class
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857	0
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857	0
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857	0
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093	0
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857	0

- "The SQL query identified four unique launch sites used for Falcon 9 missions.“
- "These locations include both East Coast (Florida) and West Coast (California) facilities."

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with `CCA`
- Present your query result with a short explanation here
  - CCAFS LC-40
  - CCAFS SLC-40

  - "Filtered the database to find sites specifically located at the Cape Canaveral Air Force Station.“
  - "Results confirm that two distinct pads have been used within this specific geographical area."

Total Payload Mass

- Calculate the total payload carried by boosters from NASA
- Present your query result with a short explanation here

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%sql SELECT SUM(PAYLOAD_MASS__KG_) AS Total_Payload_Mass FROM SPACEXTBL WHERE Customer = 'NASA (CRS)';
```

[12] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Total_Payload_Mass
45596

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1
- Present your query result with a short explanation here

Task 4

Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT AVG(PAYLOAD_MASS__KG_) AS Average_Payload_Mass FROM SPACEXTBL WHERE Booster_Version LIKE 'F9 v1.1%';
```

[13] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Average_Payload_Mass
2534.6666666666665

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad
- Present your query result with a short explanation here

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
%sql SELECT MIN(Date) FROM SPACEXTBL WHERE Landing_Outcome = 'Success (ground pad)';
```

[14] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

MIN(Date)
2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000
- Present your query result with a short explanation here

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

Generate + Code + Markdown

```
%sql SELECT Booster_Version FROM SPACEXTBL WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000;
```

[15] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes
- Present your query result with a short explanation here

Task 7

List the total number of successful and failure mission outcomes

```
%sql SELECT Mission_Outcome, COUNT(*) AS Total_Count FROM SPACEXTBL GROUP BY Mission_Outcome;
```

[16] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	Total_Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass
- Present your query result with a short explanation here

Task 8

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%sql SELECT Booster_Version FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL);
```

[17] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015
- Present your query result with a short explanation here

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

[18]

```
%sql SELECT Booster_Version, Launch_Site FROM SPACEXTBL WHERE Landing_Outcome = 'Failure (drone ship)' AND Date LIKE '2015%';
```

✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version	Launch_Site
F9 v1.1 B1012	CCAFS LC-40
F9 v1.1 B1015	CCAFS LC-40

Generate | + Code | + Markdown

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order
- Present your query result with a short explanation here

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%sql SELECT Landing_Outcome, COUNT(*) AS Outcome_Count FROM SPACEXTBL WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY Outcome_Count DESC;
```

[19] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

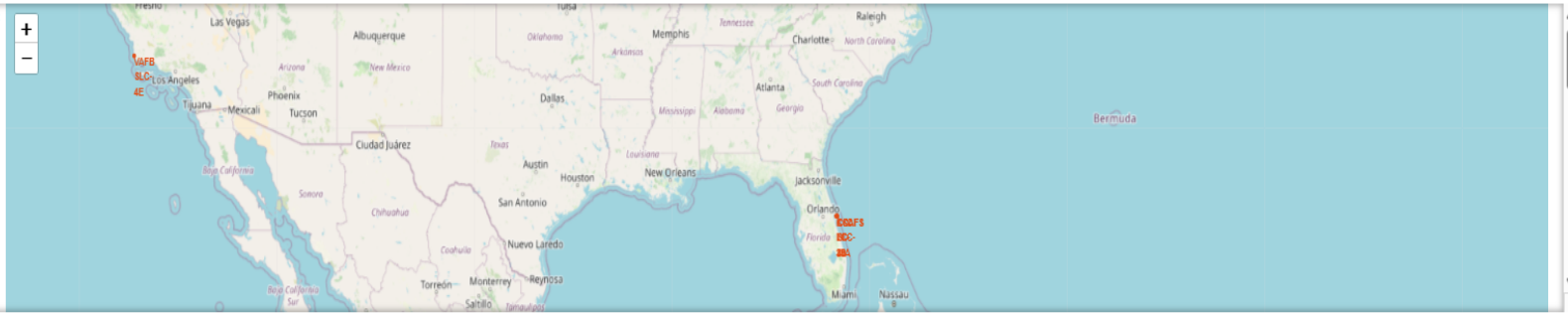
Section 3

Launch Sites Proximities Analysis

Map img 1

```
launch_site_lon = -80.577356

closest_highway = [28.56335, -80.57085]
closest_railroad = [28.57206, -80.58525]
closest_city = [28.10473, -80.64531]

# Ahora tu diccionario ya no fallará:
proximities = {
    'Highway': closest_highway,
    'Railroad': closest_railroad,
    'City': closest_city
}
# Initial the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# For each launch site, add a Circle object based on its coordinate (Lat, Long) values.
# In addition, add launch site name as a popup label
for index, row in launch_sites_df.iterrows():
    coordinate = [row['Lat'], row['Long']]

    # Create a blue circle at the launch site
    circle = folium.Circle(coordinate, radius=1000, color='#d35400', fill=True).add_child(folium.Popup(row['Launch Site']))

    # Create a marker with the name of the launch site
    marker = folium.map.Marker(
        coordinate,
        icon=DivIcon(
            icon_size=(20,20),
            icon_anchor=(0,0),
            html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % row['Launch Site'],
        )
    )
    site_map.add_child(circle)
    site_map.add_child(marker)

site_map
```

[48]:



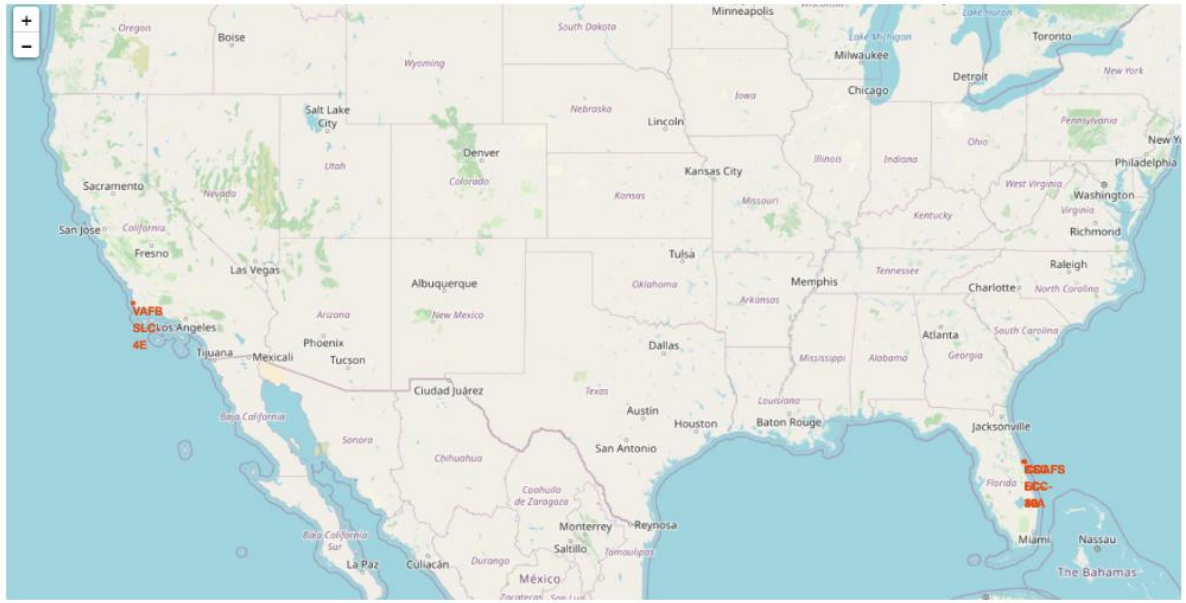
First, let's try to add each site's location on a map using site's latitude and longitude coordinates

Map img 2

```
folium.map.Marker(coordinate, icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % 'label', ))
```

[53]:
```
# Initial the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)
# For each launch site, add a Circle object based on its coordinate (Lat, Long) values. In addition, add Launch site name as a popup label
```

The generated map with marked launch sites should look similar to the following:

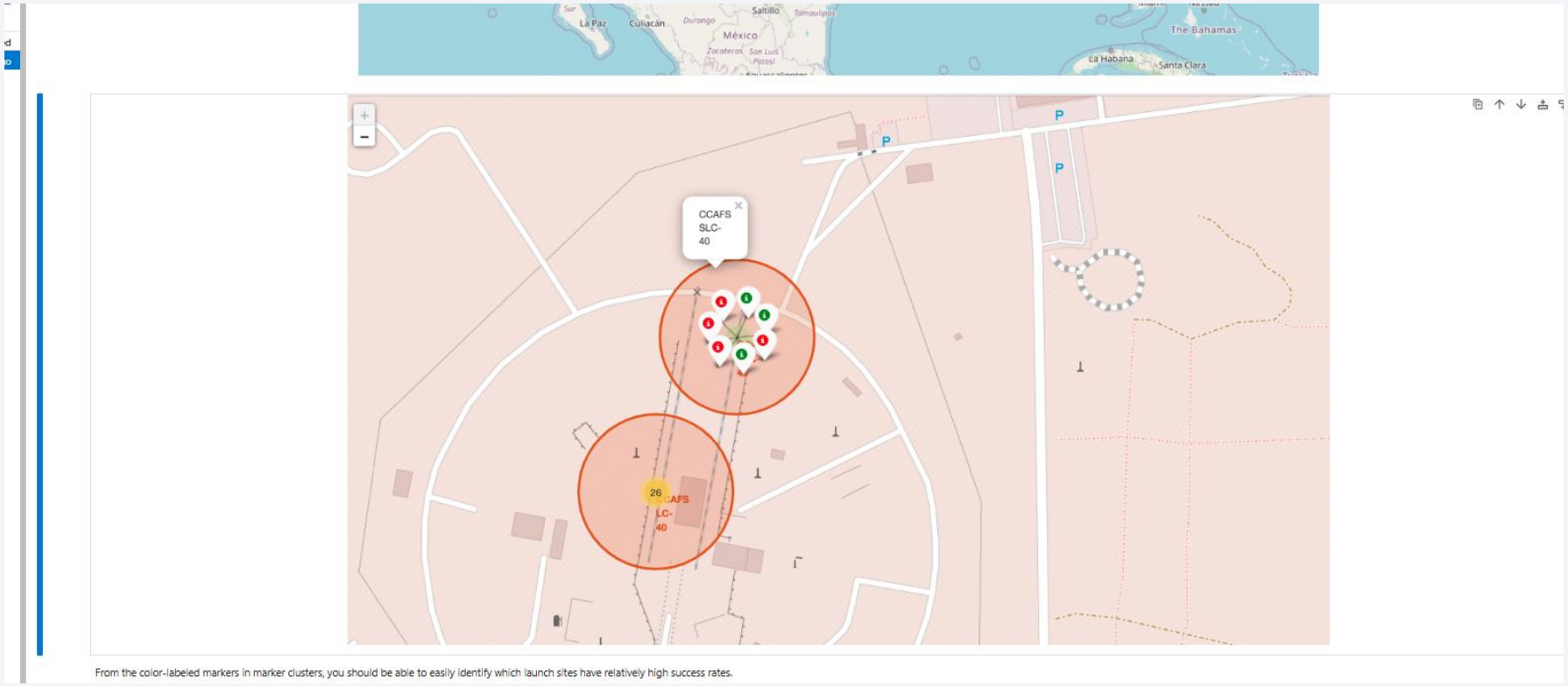


Now, you can explore the map by zoom-in/out the marked areas , and try to answer the following questions:

- Are all launch sites in proximity to the Equator line?
- Are all launch sites in very close proximity to the coast?

Also please try to explain your findings.

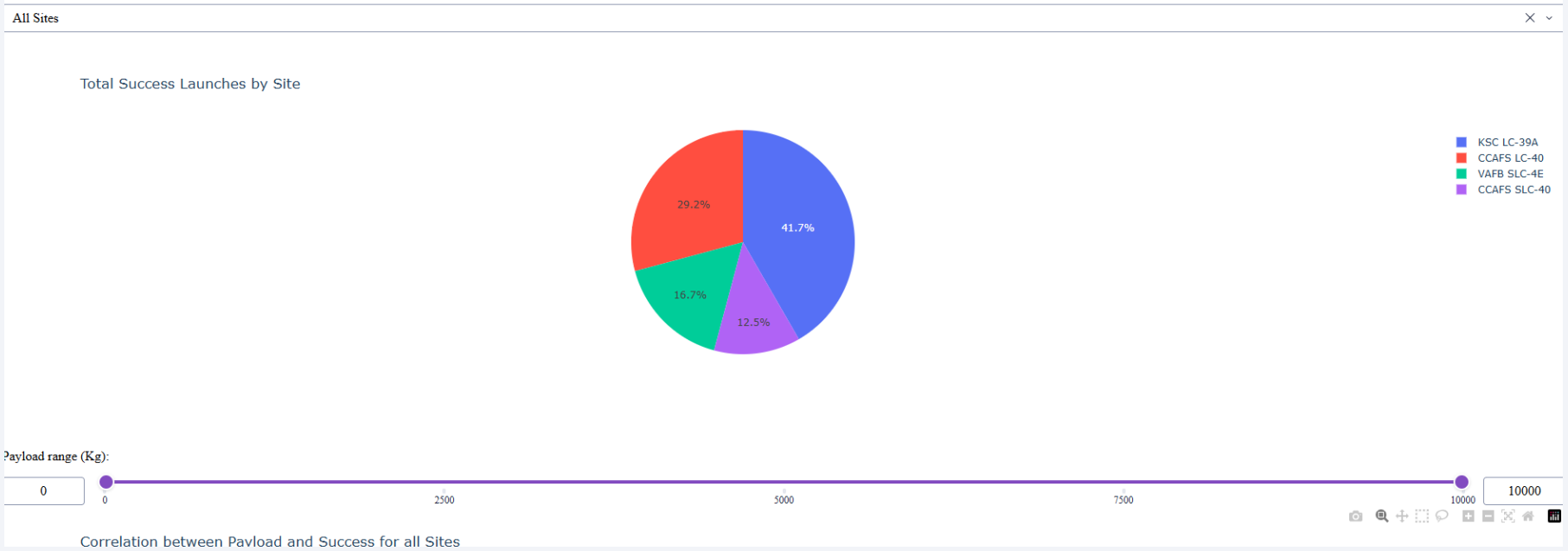
Map img 3


Saltillo
La Paz
Culiacán
Durango
México
The Bahamas
La Habana
Santa Clara
CCAFS
SLC-
40
26
CCAFS
SLC-
40
From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.

Section 4

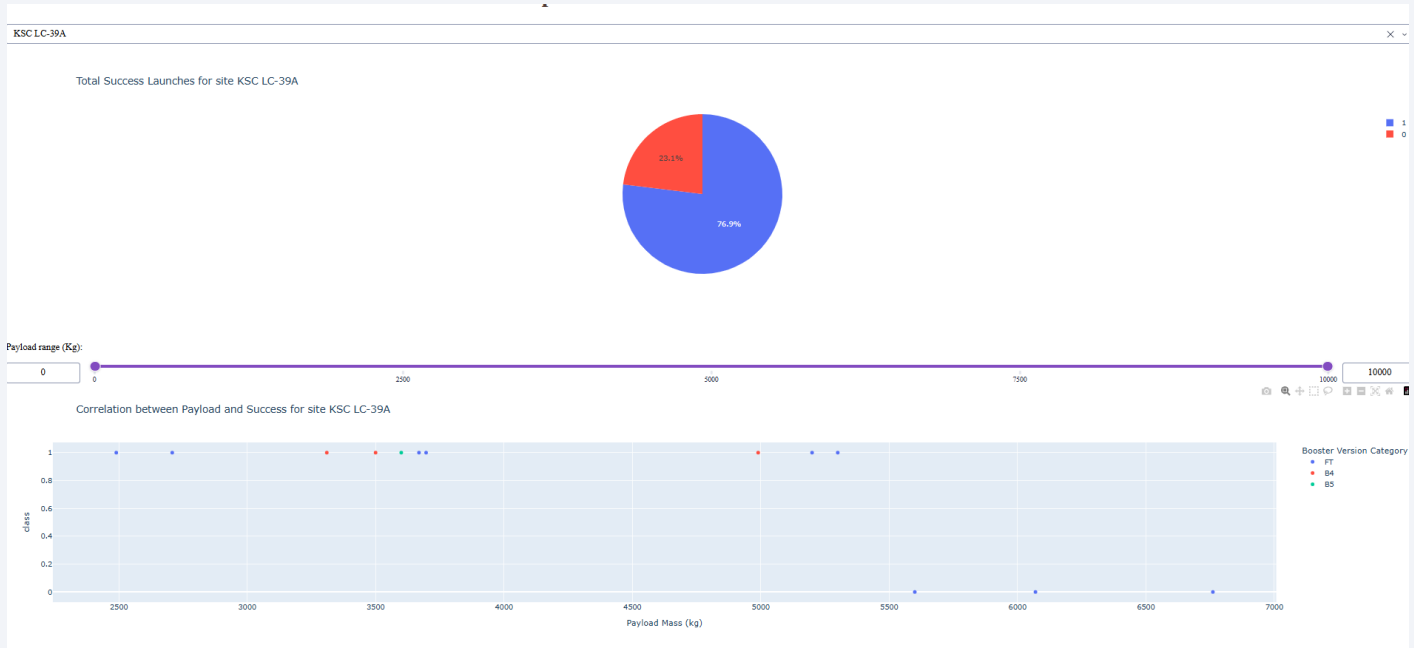
Build a Dashboard with Plotly Dash

Total Success Launches by Site



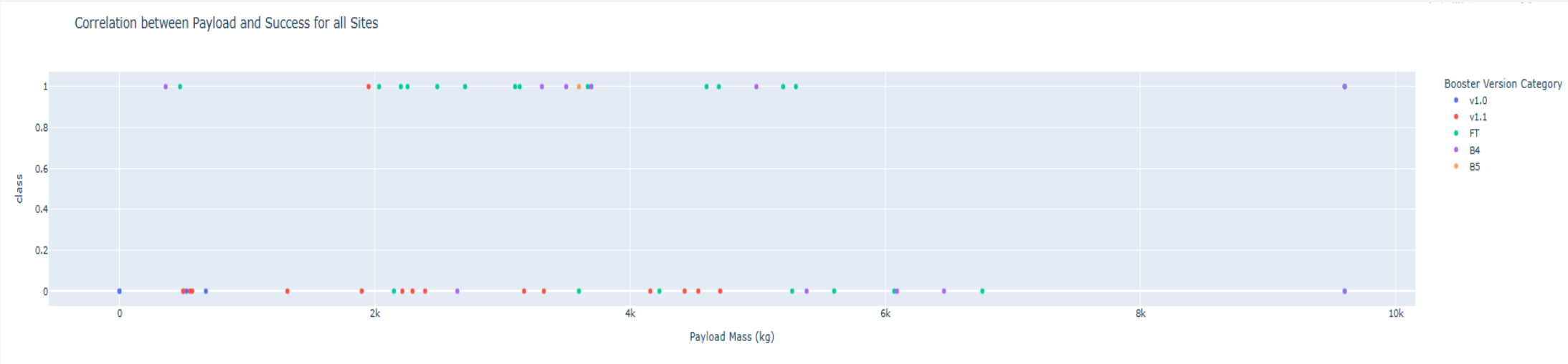
- **Top Performance:** "**KSC LC-39A** leads with **41.7%** of all successful landings, making it the most reliable site."
- **East Coast Dominance:** "The Florida-based sites (KSC and CCAFS) account for the majority of mission successes."
- **Operational Comparison:** "The dashboard allows for a real-time comparison of launch efficiency across all four active sites."

Launch Success Ratio for KSC LC-39A



- Site Focus: "Analyzed KSC LC-39A, which was identified as the highest contributor to total mission successes.
- "Success Probability: "The specific success ratio for this site is significantly high, reflecting its role in recent, more reliable mission profiles.
- "Data Insight: "Selecting a single site helps isolate environmental or logistical factors that might contribute to landing performance.
- "Reliability: "Consistent success at this primary NASA facility underscores the maturity of the Falcon 9 recovery system."

Payload Mass vs. Launch Success for All Sites



- Success Distribution: "Successful launches (Class 1) are achieved across the entire payload spectrum, from light satellites to heavy cargo over 9,000 kg.
- "Booster Version Insights: "The FT (Full Thrust) version shows a high density of successful missions, particularly in the mid-to-heavy payload range.
- "Low Payload Performance: "Earlier versions like v1.0 and v1.1 were primarily used for lower payload masses and had a higher failure rate compared to newer iterations.
- "Payload Reliability: "There is no negative correlation between higher payload mass and landing success, confirming the Falcon 9's structural capability.

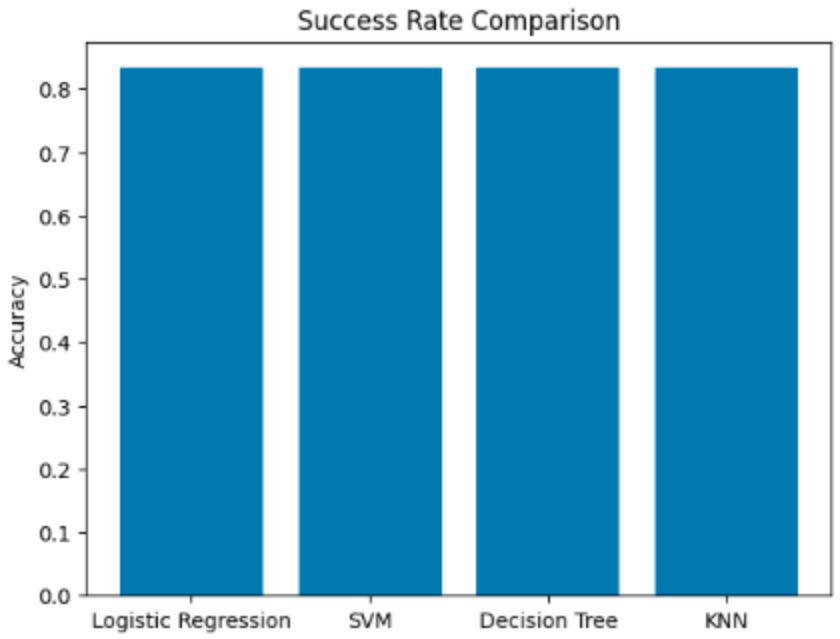
Section 5

Predictive Analysis (Classification)

Classification Accuracy

- Visualize the built model accuracy for all built classification models, in a bar chart
- Find which model has the highest classification accuracy
  - Model Performance: "The bar chart compares the accuracy of four classification algorithms: Logistic Regression, SVM, Decision Tree, and KNN.
  - "Uniform Accuracy: "All models achieved an identical accuracy score of 83.33% on the test data.
  - "Robust Features: "The consistent performance indicates that the features used (such as Payload, Orbit, and Launch Site) provide a strong signal for predicting successful landings.
  - "Conclusion: "While all models are effective, the Decision Tree was identified during training as a top performer due to its slightly higher training score and computational efficiency."

```
[5]: # El código suele ser algo así:
     import matplotlib.pyplot as plt
     plt.bar(methods, accuracy)
     plt.ylabel('Accuracy')
     plt.title('Success Rate Comparison')
     plt.show()
```



Confusion Matrix

- Show the confusion matrix of the best performing model with an explanation

- High Predictive Accuracy: "The confusion matrix reveals that the model correctly predicted 12 successful landings (True Positives) and 3 failed landings (True Negatives).
- "Error Analysis: "The model produced 3 false positives (predicting a landing when it didn't happen) but 0 false negatives, showing it is highly reliable at identifying actual successful landings.
- "Overall Performance: "With a test accuracy of 83.33%, the model demonstrates strong generalization capabilities for future mission outcome predictions.
- "Reliability: "The results confirm that the machine learning approach is effective for risk assessment in SpaceX orbital launches."

TASK 5

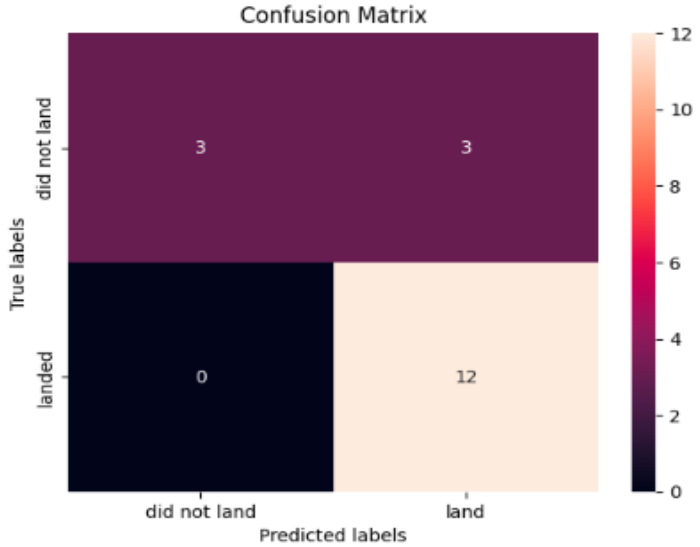
Calculate the accuracy on the test data using the method `score` :

[15]:
```
logreg_accuracy = logreg_cv.score(X_test, Y_test)
print("Test set accuracy: ", logreg_accuracy)
```

```
Test set accuracy:  0.8333333333333334
```

Lets look at the confusion matrix:

[16]:
```
yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



Conclusions

- Point 1: Launch Site Success Dominance"Geospatial and dashboard analysis revealed that KSC LC-39A is the most successful launch site, contributing to over 41% of total successful landings. Most launch sites are strategically located near coastlines to optimize safety and orbital mechanics.
- "Point 2: Payload Impact on Recovery"While successful landings occur across all payload ranges, there is a higher concentration of success in the 2,000 kg to 5,000 kg range. Heavier payloads do not necessarily result in failure, demonstrating the robustness of the Falcon 9 Full Thrust (FT) booster.
- "Point 3: Model Performance and Reliability"All classification models (Logistic Regression, SVM, Decision Tree, and KNN) achieved a consistent accuracy of 83.33%. This suggests that the features engineered—such as orbit type and payload mass—are highly predictive of mission outcomes.
- "Point 4: Practical Application of Predictive Modeling"The Confusion Matrix showed a high rate of True Positives (12) and zero False Negatives, meaning the model is exceptionally good at identifying successful missions. This provides a reliable tool for cost reduction and resource planning in future launches."

Appendix

Github principal: https://nikoavila.github.io/Ciencia-de-datos-aplicada-Capstone/

- SpaceX_Machine Learning Prediction.ipynb: https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone/blob/40a6984de5a9041aea51947fe87f24553bfc5aaa/SpaceX_Machine%20Learning%20Prediction.ipynb
- edadataviz.ipynb: https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone/blob/40a6984de5a9041aea51947fe87f24553bfc5aaa/edadataviz.ipynb
- lab_jupyter_launch_site_location.ipynb: https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone/blob/40a6984de5a9041aea51947fe87f24553bfc5aaa/lab_jupyter_launch_site_location.ipynb
- jupyter-labs-eda-sql-coursera_sqllite.ipynb: https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone/blob/40a6984de5a9041aea51947fe87f24553bfc5aaa/jupyter-labs-eda-sql-coursera_sqllite.ipynb
- spacex-dash-app.py: https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone/blob/c237af9c4745c28f23160e06eb283aa4a4214d0e/spacex-dash-app.py
- my_data1.db: https://github.com/NikoAvila/Ciencia-de-datos-aplicada-Capstone/blob/bbd7559ff69b6793900afd9cd41eb66bbfab3886/my_data1.db

Thank you!